

SELECTOR

selector

`h1 { color : red ; }`

value

property

(a) Universal Selector

– to select everything in a doc.

`* { color : red ; }`

(b) Element Selector

– selects all elements of same type

`h1 { color : red ; }`

`* h1, h3 { color : red ; }`

changes colour in both

(c) ID Selector

- Selects an element based on value of the element's id attribute

```
<h1 id = "myId" >
```

→ html (case sensitive)

```
# myId {
```

```
    property : value ;
```

```
}
```

→ css

(d) Class Selector

- Select an element based on their class attribute

```
<h1 class = "myclass" >
```

```
<button class = "myclass" >
```

→ html (case sensitive)

```
.myclass {
```

```
    property : value ;
```

```
}
```

→ css

(Practice Question)

Implement the following CSS for the given html document.

- Set font family of entire document to Courier New
- Set text color for all h1 headings to #1b74e4
- Set background color of search button to black & text to white
- Set background color of login & signup button to aqua.

```
<head>
  <meta charset="UTF-8" />
  <meta http-equiv="X-UA-Compatible" content="IE=edge" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Practice Qs</title>
  <link rel="stylesheet" href="1_Practice_Q.css" />
</head>
<body>
  <h1>Facebook</h1>
  <button class="userbtn">Log in</button>
  <button class="userbtn">Register</button>
  <hr />
  <label for="search">Search</label>
  <input placeholder="search your friends" id="search" />
  <button id="searchbtn">Search</button>
</body>
</html>
```

→ html

```
*{
  font-family: 'Courier New';
}
h1{
  color: #1b74e4;
}
#searchbtn{
  background-color: black;
  color: white;
}
.userbtn{
  background-color: aqua;
}
```

→ CSS

Facebook

Log in Register

→ o/p

Search

(e) DESCENDANT SELECTOR

- Selects all paragraphs inside divs →

`<div><p></p></div>`

```
div p {  
  color: "red";  
}
```



```
div p ul {  
  color: "red";  
}
```



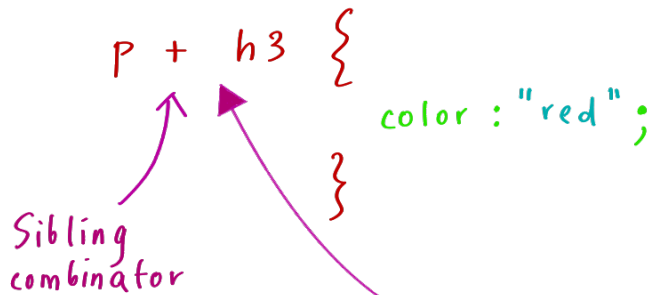
```
.myclass ul {  
  color: "red";  
}
```



Multiple selectors used by COMBINATORS:

(i) ADJACENT SIBLING COMBINATOR

eg: `h3` that comes immediately after any paragraph



```
<P></P>  
<h3></h3>  
<h3></h3>
```

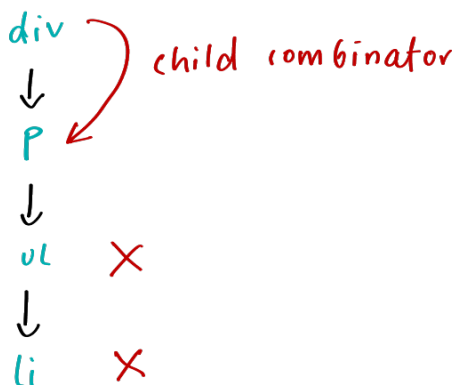
`P ↔ h3` (siblings)

`h3`
↓
`P` or `P`
↓
`h3` **X** (no heirarchy)

(ii) CHILD COMBINATOR

eg: selects all buttons that are direct children of `div`

```
<div> <P> <ul> <li> </li> </ul> </P> </div>
```



`div > P` {
`color: "red";`
}

(f) ATTRIBUTE SELECTOR

- Selects elements based on the presence or value of a given attribute

```
<input type = "text" placeholder = "username" >
```

 → html

```
input [type = "text"] {  
    color : green ;  
}
```

 → css

*

```
input [type] {  
    color : green ;  
}
```

→ Selects every type (ie : text, password,)

Practice Question

Q.) Set color for paragraphs inside div with id "myID" to green

(Ans)

```
div [id = "myID"] p {  
    color : green ;  
}
```

OR

```
#myID p {  
    color : green ;  
}
```

PSEUDO CLASS

- A keyword added to a selector that specifies a special state of the selected element.

① `h3 : hover {` → mouse hover
`property : value ;`
`}`

② `h3 : active {` → clicked state
`property : value ;`
`}`

③ `input[type = "radio "]: checked {` → checkbox selected
`property : value ;`
`}`

④ `div : nth-of-type (1) {` → select n^{th} element
`property : value ;`
`}`

↙
change property where $\text{div} = 1$ (1^{st} div)

* Similarly we can implement $(2n), (3n), \dots$

PSEUDO ELEMENTS

- keyword added to selector that lets you style a specific part of the selected element(s)

eg: ① `h1 :: first-letter {`
`color : red ;`
`}`

② `h1 :: first-line {`
`color : red ;`
`}`

③ `h1 :: selection {`
`color : red ;`
`}` → selected area by the mouse

CSS: Cascading Stylesheets

cascade : CSS cascade algo.'s job is to select CSS declarations in order to determine the correct values for CSS properties

```
h2 {  
  color : red ;  
}
```

✗ (not considered)

```
h2 {  
  color : red ;  
}
```

✓

- works on same selector
- This cascading algo even works on diff. CSS files.

eg: If 2 CSS files have same element (h2) then whatever comes later will be considered.

SELECTOR SPECIFICITY

Specificity : An algo that calculates weight that is applied to a given CSS declaration.

id > class, Attribute, pseudo-class > element pseudo-element

Ex :

<code>.myClass:hover { color: blue; }</code>		
0	2	0
id	class, attribute & pseudo-class	element & pseudo-element

= 20

<code>h2 { background-color: yellow; }</code>		
0	0	1
id	class, attribute & pseudo-class	element & pseudo-element

= 1

Here acc to specificity, 1st 20 times stronger than 2nd

Note : If specificity same then use cascading algo

INLINE SPECIFICITY

→ Writing CSS in HTML file itself

→ more specific than id

→ mostly avoided.

inline style > id > class, Attribute, pseudo-class > element pseudo-element

Practice question

For the given html & css code, what is color of h1:

```
<h1 id="greet" class="myClass">Hello!</h1>
```

```
h1 {  
  color: pink;  
}  
  
#greet {  
  color: yellow;  
}  
  
.myClass {  
  color: brown;  
}
```

Ans → yellow
(id → most prior)

Practice Qs

For the given html & css code, what is bg-color of p:

```
<div>  
  <p class="para">This is a paragraph.</p>  
</div>
```

```
div p {  
  background-color: black;  
}  
  
.para {  
  background-color: blue;  
}  
  
p {  
  background-color: green;  
}
```

→ 0 0 2

2

→ 0 1 0

10

→ 0 0 1

1

(Ans) blue

pragyanprayasjena@gmail.com

! IMPORTANT

– To show the most specific thing in document

```
h2 {  
    background-color : blue    !important ;  
}
```

→ even more specificity than inline.

INHERITANCE IN CSS

- In CSS, certain properties applied to a parent or grandparent element can automatically pass down to child elements. This inheritance flows in one direction only—from parent to child—meaning child elements do not pass their styles up to parents or grandparents.
- However, it's important to note that only specific properties—mainly those related to text and fonts, like color, font-size, and line-height—inherit by default. Properties related to box models, spacing, or layout (like margin, padding, and border) do not.
- Some elements, such as buttons and form inputs, often do not inherit styles even for inheritable properties because of browser defaults or their special nature. In such cases, if you want them to inherit styles like color or font, you need to explicitly apply the inherit value in your CSS. For example:

```
button {  
    color : inherit ;  
}
```